iOS Speech and Pronunciation: Platform Analysis

What we can solve on web, what we can't, and what we need to measure before deciding

Prepared by: Harshan **Date:** 28 July 2026 **Status:** Analysis complete. Measurement pending. No platform recommendation until §15 is done.

Summary

We are hitting speech and microphone problems on iPhone and Mac, and the question has come up whether we need a native app. This document separates what is actually going on.

There are three distinct problems, and mixing them leads to the wrong decision.

Problem	Cause	Fixable on web?
Reliability — speech stops working, wrong browser, timeouts	The Web Speech API, not the microphone	**Yes**
Audio quality — iOS processes microphone audio before we see it	iOS audio session, below the browser	**No** — but impact unmeasured
Scoring — our pronunciation feedback may be misjudging learners	Our own scoring approach	**Yes**, and native would not fix it

Two questions, two very different confidence levels:

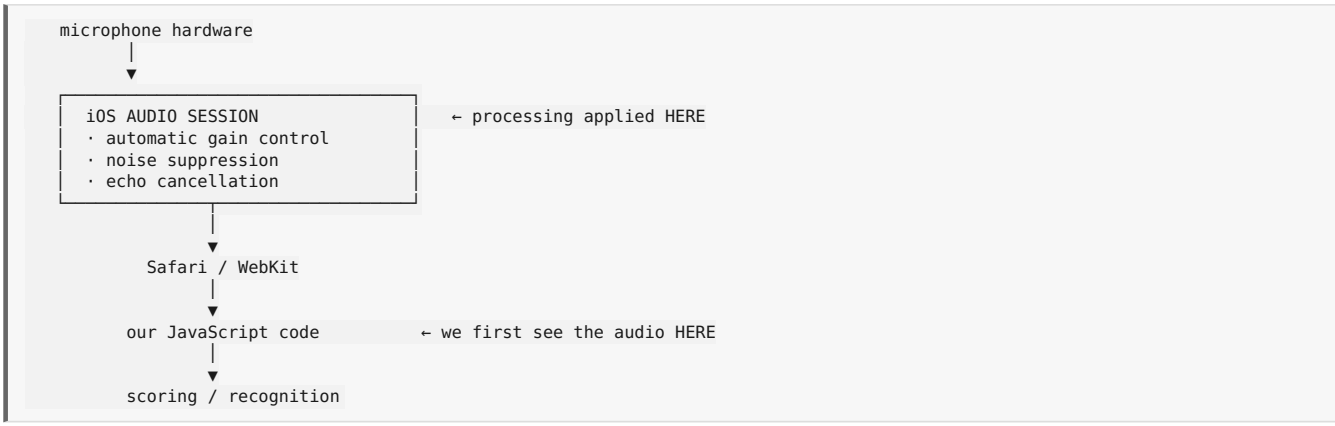
Question	Answer	Confidence
Can we stop iOS processing our microphone audio in a web app?	**No.** No mechanism exists.	**High — settled**
Does that processing actually degrade our pronunciation scoring?	**We don't know.** Plausible, but unmeasured.	**Low**

"*We can't remove the processing*" and "*the feature won't work*" are different statements. It is easy to slide from the first to the second and conclude we need an app. Of four possible test outcomes, only one would justify that.

Ask: approximately eight days to measure before we commit either way.

Part 1 — The platform constraint

1. Where the processing happens



By the time our code receives a single sample, the processing has already been applied. We are downstream of it. There is nothing to switch off at our layer because the change happened two layers above us.

All three are correct decisions for phone calls, which is what iOS optimises for by default.

2. Why the web app cannot reach it

The audio session belongs to the app that owns it. On iOS that app is Safari. Our page is content running inside Safari — it does not own the audio session and cannot configure it.

`getUserMedia` constraints are a request, not a command. When we write `noiseSuppression: false`, we ask the browser to prefer unprocessed audio. The browser can honour that only within what its own audio session permits. If Safari's session has voice processing enabled, our request cannot override it.

This is a deliberate sandbox boundary. The audio session is a shared system resource affecting call audio, routing to speakers or Bluetooth, and other apps' behaviour. Allowing any web page to reconfigure it would be a real security and stability problem. Apple's position is defensible even though it is inconvenient for us.

Every iOS browser inherits this. Chrome, Edge, Firefox and in-app browsers on iOS all run on WebKit. Switching browsers changes nothing.

One caveat to verify. An emerging web standard — the Audio Session API — has begun appearing in Safari, giving pages some control over audio session *type*. As far as we know it does not expose voice-processing control, but this is worth 30 minutes against current WebKit documentation before we treat the constraint as permanent.

3. The accurate way to state this

We should not say "*iOS Safari cannot capture microphone audio*." That is wrong and will mislead anyone who repeats it.

The accurate statement is:

> **iOS Safari can capture microphone audio. A web app does not have the same low-level audio-session control that a native iOS app has.**

That distinction matters, because the first version implies we are blocked and the second describes a specific, bounded limitation.

Part 2 — What is actually failing today

4. Most current failures are Web Speech, not the microphone

What the user sees	Likely cause	Fixed by
Works in Safari, dead in Chrome or an in-app browser	`SpeechRecognition` is not exposed in WKWebView, which all non-Safari iOS browsers use	Moving off Web Speech
Stops after 30–60 seconds	Safari's recognition timeout; continuous mode unreliable on iOS	Moving off Web Speech
Worked this morning, stopped this afternoon	Apple's per-device recognition request quota	Moving off Web Speech
A chime before every attempt	System sound tied to speech recognition	Moving off Web Speech
Permission prompt every session	Safari permission lifetime	Not fixable; expected
Silent failure after a call, Siri, or an alarm	Audio session interruption ended the capture track	Detectable and recoverable in code
Nothing happens on tap, no error	AudioContext suspended; iOS requires a user gesture	Fixable in code
Scores feel wrong for accented speakers	Scoring approach, not capture	§16 — a different problem

The first four disappear when we stop using the Web Speech API. `getUserMedia` works in WKWebView, has no quota, no session timeout, and no chime. None of this requires a native app.

What we need from users to diagnose the rest. "It doesn't work on my iPhone" is the most common report and the least actionable. Per report we want: which browser (Safari, Chrome, or opened inside another app), iOS version, what exactly happened, and whether it reproduces. Four or five reports at that detail would tell us which rows we are actually in.

5. What the web app can solve

Area	Web solution
Microphone permission	Four-state handling with per-browser recovery guidance
Audio capture quality	AudioWorklet with raw PCM, rather than MediaRecorder and container formats
Non-Safari iOS browsers	Move off Web Speech to `getUserMedia`
Timeouts and quotas	Same — these are Web Speech behaviours
Speech boundary detection	Voice activity detection in the browser
Noisy recordings	Signal-to-noise checks before scoring, instead of scoring bad audio
Microphone interruptions	Detect track-ended and session interruptions; recover or report clearly
Suspended audio context	Detect, and prompt for the gesture iOS requires
Silent failures	Explicit error states instead of an interface that says "listening" while capturing nothing
Pronunciation scoring	A proper assessment service returning phoneme-level detail
Vendor lock-in	Normalise scores through our own gateway so the vendor stays replaceable
Platform score differences	Calibrate, if measurement shows a consistent offset

---

Part 3 — What native would and would not change

6. What native gives us

Capability	Web app	Native app
Disable gain control, noise suppression, echo cancellation	**No**	**Yes** — `AVAudioSession` measurement mode
Set preferred sample rate and buffer size	Limited	Yes
Microphone permission persists across sessions	No	Yes
Capture while backgrounded or screen locked	No	Yes
On-device speech frameworks	No	Yes
Control audio routing	No	Yes

The first row is the substantive one. The rest are conveniences.

7. What native does not give us

Our problem	Fixed by native?	Why
Speech fails in non-Safari iOS browsers	**No**	Web Speech, not the microphone. Fixed on web.
Recognition timeout, quota, chime	**No**	Same cause, same fix
Speech-to-text accuracy	**No**	We call the same cloud API from either platform
Text-to-speech	**No**	Playback, not capture
Higher error rates for accented speakers	**No**	A property of the recognition model and its training data
Cloud API latency	**No**	Same network, same service
Classroom noise and other speakers	**No**	§11 — a physical problem, not a platform one
Pronunciation scoring built on transcript matching	**No**	Our own code. It ships unchanged into an app.
Unprocessed audio for scoring	**Yes**	The one genuine gain — and its value is unmeasured

Two of ten, and the only significant one is unproven.

8. Why learning products choose app-first

Competitors being app-first is often cited as evidence. Before reading it that way, here are the reasons these companies would have gone native regardless.

Driver	Why it favours an app	Audio-related?
Daily habit and streaks	Push notifications are the retention engine of every language app. On iOS, web push requires adding the site to the home screen first — most users never do.	No
Home-screen presence	An icon is a daily prompt to return; a bookmark is not	No
App-store discovery	Category browsing, search ranking, editorial features	No
Subscription conversion	In-app purchase converts better on mobile, despite the revenue share	No
Offline use	Commute and travel learning	No
Background and lock-screen audio	Listening lessons with the screen off	Partly — playback
Gamified interface performance	Animation-heavy streak and reward interfaces	No
Attribution and analytics	Mobile measurement tooling is more mature	No
Permission persistence	Microphone granted once, not re-prompted	Marginally
Unprocessed audio capture	`AVAudioSession` measurement mode	**Yes — the only one**

One of ten concerns audio. When a competitor ships an app, that decision is heavily overdetermined, and reading it as evidence about capture is not sound.

The reverse also holds: if any of the first five rows are strong reasons for *us*, an app may be worth building — but as a growth decision argued on its own terms, not as a technical necessity.

9. What competitors actually do

Nobody publishes their audio capture problems. The following is inference from public product behaviour, not knowledge of

internal decisions.

Company	Observable behaviour	Plausible reading
ELSA	Native; built their own assessment models rather than using a vendor API	Suggests hosted scorers were insufficient — a finding about *models*, not capture
Duolingo	Speaking exercises are notably forgiving; rarely phoneme-level feedback	Looks like a deliberate choice to set the bar low enough that false negatives don't frustrate learners
Speechling	Native; heavy use of human coach review alongside automation	Human-in-the-loop as a hedge against unreliable automated scoring
Microsoft Reading Coach	Browser-based pronunciation assessment at education scale	Our most relevant counter-example: same vendor as our intended scorer, shipped on web
Speechace, Azure	Both sell pronunciation assessment with documented web integration	Vendors don't maintain web SDKs for a channel that doesn't work

Where the industry's effort has actually gone: not into fixing capture, but into **models that tolerate imperfect audio**. Whisper, wav2vec2 and successors were trained on messy real-world recordings precisely because clean consumer capture is unattainable. That is the strongest reason to think our scorer may handle processed iPhone audio without difficulty.

A fifteen-minute check worth doing. Azure and Speechace publish supported-platform documentation. Whether they consider iOS web a supported target is public record, and worth more than any inference above.

10. The middle option: Capacitor

Native does not have to mean rewriting the product.

Capacitor packages our existing React web app as an installable iOS and Android app while giving it native device access through plugins — including audio capture. The web codebase stays as it is.

	Web only	Capacitor wrapper	Full native
React codebase reused	Yes	**Yes**	No
Unprocessed audio on iOS	No	**Yes**, via plugin	Yes
Persistent mic permission	No	Yes	Yes
App store presence	No	Yes	Yes
Codebases to maintain	1	1 + thin shell	2–3
Users must install	No	Yes	Yes
Store review per release	No	Yes	Yes
Build effort	—	Weeks	Months

The web app would still need to exist for users who don't install, so this adds a platform rather than replacing one.

If measurement shows the audio problem is real, this is almost certainly the answer — it delivers the one thing native offers without the cost that makes full native unattractive.

Part 4 — Problems neither platform solves

11. Environment: noise and other speakers

This may matter more than audio processing does.

A microphone captures whatever is in the room. In a classroom that means other learners, a teacher, and ambient noise — and if another person is speaking closer to the microphone than our learner, their voice is the louder signal.

What software can do:

Technique	What it handles
Voice activity detection	Skips silence; avoids scoring non-speech
Signal-to-noise gating	Rejects a recording as too noisy *before* scoring, rather than returning a bad score
Target-utterance alignment	Confirms the audio contains the expected words before scoring against them
Speaker verification	Could confirm the audio matches the enrolled learner — but this is voice biometrics, substantially larger than the three above, with its own privacy and regulatory weight

What software cannot do: reliably separate a second person speaking close to the microphone. Source separation at that quality is not a solved consumer problem.

For scored assessment the practical answer is hardware. A close-talking or boom headset puts the learner's mouth near the microphone and everything else far from it — solving by physics what cannot be solved by processing. For any scored assessment this should be a stated requirement, not a suggestion.

This affects web and native identically.

12. Network

Condition	Effect	Mitigation
Slow or unstable network	Latency — the learner waits longer. Scores are unaffected.	Record locally, upload with retry, show progress honestly
No network	No score possible	Only an on-device model changes this — a large undertaking

One point worth being precise about. Streaming audio over a WebSocket reduces waiting for *continuous* recognition — free speech, dictation, conversation — because results arrive while the person is still talking.

It does not help pronunciation assessment. Scoring a bounded utterance against a known reference text requires the complete recording; the scorer cannot begin until the learner finishes. For that path a plain upload is correct, and streaming adds complexity without reducing latency.

13. Two architectural paths, kept separate

```
 SCORED EXERCISE  (bounded utterance, known reference text)
Mic → AudioWorklet → PCM → VAD / SNR check → POST → Gateway → Azure → score

FREE SPEECH  (dictation, conversation practice — future)
Mic → AudioWorklet → PCM → VAD → WebSocket → Gateway → Azure → live transcript
```

These should not share one pipeline. The scored path is batch and needs no WebSocket at all — which means our highest-value work requires none of the streaming infrastructure.

Part 5 — What we don't know, and how to find out

14. Why the processing may not actually matter

The reasoning that iOS processing degrades scoring is plausible but untested. There are equally reasonable arguments the other way:

- **Gain control compresses volume, not frequency content.** Phoneme identity lives mostly in formant structure, which largely survives gain changes. What it does affect is relative loudness between syllables, which carries stress.
- **Noise suppression is tuned to preserve speech intelligibility** — it is actively trying to protect the frequencies phonemes live in.
- **Modern speech models are trained on messy consumer audio,** including processed phone recordings, precisely because clean capture is unattainable at scale.
- **Any degradation is likely consistent.** iOS applies the same processing every time, and a consistent shift is a calibration constant, not a broken feature.

The risk worth worrying about is not "iPhone scores are wrong." It is that iPhone and desktop may produce *different* scores for identical pronunciation quality — so a learner switching devices sees their score move for no reason they can perceive. That is a consistency problem, and consistency problems are usually fixable with per-platform calibration.

15. The measurement: three tests, eight days

Test A — What does the browser report? *(1 hour)* Ask for processing off, read back what was granted. On Safari this often reports nothing, which is inconclusive rather than a failure — hence Test B.

Test B — Is processing applied anyway? *(half a day)* A diagnostic page measures three things directly:

Measurement	Reveals
Noise floor over 4 seconds of silence	Noise suppression leaves digital silence; a real room never gets that quiet
Level gap between a whispered and a loud take	Gain control compresses this gap
High-frequency energy in a sustained "sss"	An "s" lives above 4 kHz — the band phoneme scoring reads

Run on **iPhone Safari, iPad Safari, Mac Safari, and Mac Chrome as the control.** iPhone, iPad and Mac have different audio environments and must be tested separately rather than assumed identical.

If Mac Safari is clean and iPhone is processed, we have confirmed an iOS audio-session property — expected, not a bug, and calibration is the path. If both are processed, it is a WebKit-level issue worth checking against newer Safari releases first.

Test C — Does the scorer actually work? *(one week — the real test)*

Set	Content	~n	Expected
A	Target words pronounced *acceptably* by speakers with strong accents	40	High scores
B	The same words *deliberately mispronounced* by fluent speakers	40	Low scores

Run through the scorer on iPhone and desktop, and against our current live feature for comparison.

Outcome	Meaning	Action
Sets separate cleanly on both platforms	The scorer works; iOS processing is a footnote	Build on web, one threshold
Sets separate, iPhone consistently lower	Real but deterministic bias	Build on web, per-platform calibration
Sets separate on desktop, unreliable on iPhone	Genuine platform problem	**Now** evaluate Capacitor, with evidence
Sets don't separate anywhere	The scorer isn't the answer	Stop; investigate before building

One outcome in four points at a native layer. We currently have no evidence which one we are in.

16. The separate issue, and the more urgent one

Independent of platform: our shipped pronunciation feature is built on the browser's Web Speech API, which returns a transcript and a confidence number and **no phoneme-level data at all**.

The score is therefore inferred from whether the recognizer returned the expected word. If so, it fails in both directions:

- **False negative** — a learner with a strong accent pronounces acceptably, the recognizer fails to return the word, the learner is marked wrong. Every one of our users is a second-language speaker, so this would hit exactly the population the feature exists for.
- **False positive** — a learner mispronounces badly, but the recognizer's language model supplies the expected word anyway, and the learner is told they were correct.

This is a strong suspicion, not a confirmed finding, and Test C measures it directly.

It matters here because native would not fix it. The same scoring approach in an app produces the same feedback.

---

17. Fact and assumption

Stated plainly, so nobody carries the wrong confidence into a decision.

Statement	Status
iOS applies voice processing before our code sees the audio	**Fact**
A web app cannot disable it; the audio session belongs to Safari	**Fact** — with one emerging API worth checking
A native or Capacitor app can disable it	**Fact**
All iOS browsers use WebKit and behave identically here	**Fact**
Web Speech provides no phoneme data	**Fact**
`SpeechRecognition` is unavailable in WKWebView	**Fact**
Recognition accuracy and accent bias are identical on web and native	**Fact**
Classroom noise and co-located speakers are unaffected by platform choice	**Fact**
Software cannot reliably separate a second speaker close to the mic	**Fact**
Streaming reduces pronunciation-scoring latency	**False** — scoring needs the complete utterance
Most current failures come from Web Speech, not the microphone	**Likely** — confirm against real reports
Modern speech models tolerate processed consumer audio	**Well established in the field**
Browser-based pronunciation assessment ships in production elsewhere	**Likely** — verify before quoting
Competitors are app-first *because of* audio constraints	**Weak inference** — almost certainly multi-causal
iOS processing meaningfully degrades our pronunciation scores	**Unknown — this is what we must measure**
Our current feature misjudges accented learners	**Strong suspicion — testable in a week**

Everything in the last two rows becomes known after Test C. Nothing in this document should be used to justify a build decision or a native-app decision before that.

---

18. Recommendation

1. **Check Azure's and Speechace's supported-platform documentation** for iOS web — fifteen minutes, and it beats inference.

2. **Collect four or five detailed failure reports** using the fields in §4 — costs nothing, and may resolve the reliability question outright.
3. **Run Tests A and B** across iPhone, iPad, Mac Safari and Mac Chrome — half a day.
4. **Start recruiting speakers for the Test C fixture in parallel** — the recording is the slow part.
5. **Run Test C** on both platforms, against both the current feature and the assessment service.
6. **Decide from §15's table.**

In parallel and regardless of the outcome, the work in §5 is worth doing: moving off Web Speech to our own capture pipeline fixes the majority of what users are reporting today, on every platform.

Eight days either unblocks the build or prevents us from building the wrong thing. Both are worth the time.

Browser and platform behaviour changes between iOS releases. The constraints described here should be re-verified against the current iOS version before any final decision. Competitor and vendor examples should be checked against current documentation before being quoted outside the team.